

IBM Intern Project
UPS Watson Knowledge Graph
Yannick Tanyi

The Real-World Problem

UPS runs a large cloud platform (Watsonx / Cloud Pak for Data) for enterprise AI services like voice assistants and digital chat. Over time, when something breaks, engineers write an RCA (what broke, why, and the fix), and when they want to make a controlled change, they write a CR (what they're changing, who approved it, rollback plan).

Looking at the actual documents: there's an RCA about voice calls dropping because certificate renewal wasn't coordinated across teams, a CR about zen-minio running out of memory, RCAs about response time spikes, OSD disk issues, etc.

The problem is simple: these documents just pile up in folders. When a new incident hits, engineers have no way to know:

- "Has zen-minio had memory issues before?"
- "Which team approved the last cert change that touched Voice Gateway?"
- "Are these two incidents related?"

You're starting from zero every time.

Why Basic RAG Isn't Enough

A basic RAG system (the standard "dump documents into a search engine" approach) would let you search by keywords. But it misses connections. For example:

- The CPD cert RCA and the Voice Gateway outage are deeply connected — the cert update on CPD routes broke Voice GW because they share a trust store
- The zen-minio CR and any RCA about memory pressure are related — one is the fix, the other explains the symptom

Basic search finds words. It doesn't know zen-minio + IBM EM Team + memory alerts are one story, or that certificate + CPD + Voice Gateway + trust store are another.

The 5-Phase Pipeline

Think of it like building a map of a city by reading 294 local newspaper articles

Phase 1 — Read each article independently

Extract the important nouns from every document in isolation: service names (zen-minio, Voice Gateway, CPD), teams (IBM EM, UPS Auth), clusters (PROD East, DEV), actions (certificate renewal, memory limit increase), etc. No document sees another yet — this prevents the first documents processed from "defining" the vocabulary.

Phase 2 — Unify the vocabulary globally

Across all documents, "IBM EM Team", "IBM EM", and "EM team" are the same thing. This phase normalizes all those entities into one canonical list. Without this step, you'd get a fragmented graph where the same concept appears as multiple disconnected nodes.

Phase 3 — Find candidate pairs efficiently

You have ~294 documents. Comparing every pair would be 43,000+ comparisons — too slow. Instead, the system uses the shared entities to filter: two documents that both mention "CPD routes", "certificate", and "Voice Gateway" are likely related. Rare shared entities (like a specific cluster name) count more than common ones (like "IBM"). This collapses 43,000 pairs down to maybe a few hundred candidates.

Phase 4 — LLM judges each candidate pair

For each candidate pair, the LLM reads both documents and decides: does Document A EXTEND Document B (adds more detail to the same story)? Does it CONTRADICT it? SUPPORT it? This is where the semantic understanding happens — the LLM can see that the CPD cert CR and the CPD cert RCA are two sides of the same event.

Phase 5 — Write the graph to Neo4j

Only high-confidence, high-strength relationships survive. Each document becomes a node (stored as a filepath, not a summary), and relationships are typed edges (EXTENDS, SUPPORTS, CONTRADICTS). A cap on connections per node prevents one "hub" document from dominating everything.

What This Gives You at Query Time

When an engineer asks: "Did zen-minio have memory problems before and what was done about it?", the system doesn't just keyword-search — it:

1. Finds the zen-minio CR node in the graph
2. Traverses its relationships to find connected RCAs, related CRs, approver histories
3. Synthesizes an answer across multiple documents
4. Cites the exact source files so the engineer can go read the originals

The knowledge graph means the answer is aware of context — who approved what, which team was involved, what cluster was affected — not just which documents contain the word "zen-minio."

Feedback on this System

Issue 1: Document as Nodes (Flat Graph)

Here's the full journey entities take through this pipeline:

Phase 1 — the LLM reads each document and extracts a list of entity strings:

```
entities: ["zen-minio (Technology)", "IBM EM Team (Organization)", "memory limit (Concept)"]
```

Phase 2 — those strings get normalized and the *build_canonical_index* creates this:

```
canonical_index = {  
    "zen-minio": {  
        "document_hashes": ["hash_of_CR", "hash_of_RCA"],  
        "mention_count": 5  
    }  
}
```

Phase 2 literally has a complete map: entity → list of every document that mentions it.

Then, Phase 5 — *create_graph* does this:

```
self.neo4j.create_document_node(  
    doc_hash=doc_hash,  
    filepath=doc_data["filepath"],  
    entities=extraction["entities"], # <-- stored as a text LIST on the node  
    topics=extraction["topics"],  
    stance=extraction["stance"]  
)
```

The entities become a text property on the Document node. Not a separate node. Not a traversable connection. Just a list of strings stapled onto the document like metadata.

And the *canonical_index* — the perfect entity-to-document map Phase 2 built — is used only for Phase 3 filtering and then never touched again. It is never passed to *create_graph*. It never becomes Neo4j structure.

So what you end up with in Neo4j is this:

```
Document {filepath: "zen-minio CR", entities: ["zen-minio", "IBM EM", ...]}
      |
      --EXTENDS-->
      |
Document {filepath: "Memory RCA", entities: ["zen-minio", "memory", ...]}
```

Now ask: "What other documents involve zen-minio?"

You cannot traverse to a zen-minio node — it doesn't exist. The only option is to scan every Document node, read its entities list, and check if "zen-minio" is in there. That's a property scan, not graph traversal. You have Neo4j running a full scan every time, which is exactly what a regular database would do — there's no graph advantage being used.

Why did you land here? Because the simpler design is "entities belong to documents." It's true — they do. But a real knowledge graph says "entities also exist independently, and documents reference them." You need both. Right now you only have the first half.

Issue 2: Temporal Dimension

Look at the Phase 1 extraction prompt — it asks the LLM to extract exactly three things:

```
{
  "entities": [...],
  "topics": [...],
  "stance": "descriptive / argumentative / instructional / analytical / narrative"
}
```

No date field. Not asked for, not extracted, not stored.

And Phase 5 *create_document_node* stores: doc_hash, filepath, entities, topics, stance. Still no date. The relationship edges Phase 5 writes to Neo4j have: *relationship_type*, *strength*, *description*, *directionality*, *confidence*. Still no date.

Every CR and RCA in your docs folder has a date at the top. The zen-minio CR says February 13, 2025. The CPD cert RCA says November 10, 2025. That information is in the text that Phase 1 reads — it just never gets captured.

The result: the graph exists in a permanent "now." Every relationship is equally timeless. The edge between the CPD cert CR and the CPD cert RCA says EXTENDS with strength 8 — but the graph has no idea that the CR happened first, that it was executed, and that the RCA was written because the CR caused an outage.

So you can never answer the most important incident management question: "What changed right before this broke?" The graph can tell you two documents are related. It cannot tell you which one came first or that one caused the other.

Issue 3: Whole Documents at Query Time

Phase 4 confirms exactly how documents are read:

```
doc1_content = self.read_document(candidate["filepath1"])
doc2_content = self.read_document(candidate["filepath2"])

def read_document(self, filepath: str) -> str:
    full_path = config.DOCUMENTS_DIR / filepath
    return full_path.read_text(encoding='utf-8')
```

That's the entire file. Every word. For Phase 4 graph building, this makes sense — the LLM needs full context to judge whether two documents have a meaningful relationship.

The problem is at query time. When an engineer asks "what caused the voice outage in November 2025?", the query engine retrieves the relevant documents and presumably feeds their full text to Granite to generate an answer. A full RCA might be 800 words. Pull 4 relevant documents and you're feeding 3,200 words to an 8B model.

The actual answer — "the cert update on CPD routes wasn't coordinated with the Voice Gateway trust store" — is one sentence buried in the middle. The model has to process the entire surrounding context (approval chains, implementation steps, backout plans, lessons learned) just to find that one sentence. At 8B parameters, the further the relevant sentence is from the end of a long context, the worse the answer quality gets.

Issue 4: Candidate Filtering by Entity Overlap

Phase 3's actual filter gate from the code:

```
if len(shared_entities) >= config.MIN_ENTITY_OVERLAP:
    candidates.append({...})
```

This is a raw count of shared canonical strings. If the count is below the threshold, this document pair is permanently rejected and Phase 4 (the LLM) never sees them.

Now look at what Phase 1 extracts. Its entity format is "*EntityName (Type)*" — so the same underlying thing could come out as:

- "*zen-minio (Technology)*" from one document
- "*zen-minio (Concept)*" from another

Phase 2's normalization might or might not unify those — it depends on whether the LLM in that batch happens to group them. If it doesn't, they count as different canonical strings and reduce the shared entity count.

More importantly: Phase 1's extraction is purely entity names. If one RCA describes "the object storage service exhausting its memory allocation" and another CR says "zen-minio memory limit increase" — Phase 1 might not

extract "zen-minio" from the first one at all, because the document never uses that name. Zero shared entities. Filtered out forever.

The weighted IDF score that Phase 3 computes does not rescue these pairs. It only sorts candidates that already passed the count gate. The smarter LLM in Phase 4 — which would recognize these are the same story — never gets to see the pair because Phase 3 already blocked it.